context-free-grammer

$G = (V, T, P, S)$ → Production Rules, Starting variable

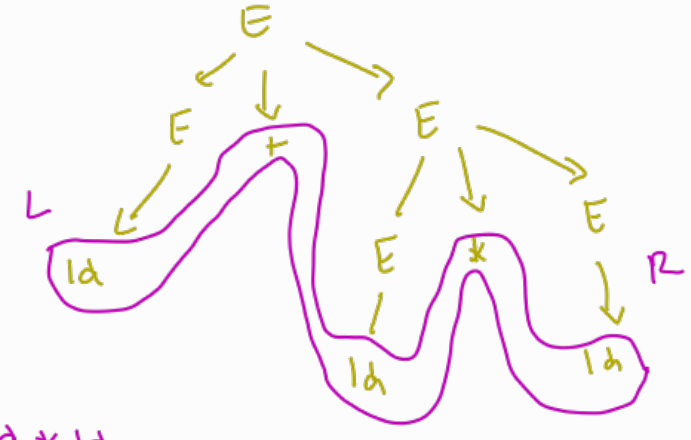
$V$ → set of variables

$T$ → set of terminals / lowercase letters

Variable e.g $A \rightarrow aB$

A is a variable where I can substitue aB.

e.g $E \rightarrow E+E \mid E*E \mid E \mid id$ obtain id + id * id




→ id + id * id

---

e.g Any no of a's $a^*$ $\Sigma = \{\Lambda, a, aa \ldots\}$

$S \rightarrow \Lambda$

$S \rightarrow aS$

e.g Any no of á's and b's. $\Sigma = \{\Lambda, a, b, ab, ba, aa \ldots\}$

RE = $(a+b)^*$

$S \rightarrow \Lambda$

$S \rightarrow aS$

$S \rightarrow bS$

→ Any no of a's and b's

eg atleast 2 a's $\Sigma = \{aa, aaa, aaaabb \ldots\}$

$(a+b)^* a (a+b)^* a (a+b)^*$

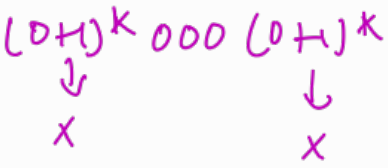
X X X

$S \rightarrow XaXaX$

$X \rightarrow aS \quad X \rightarrow \Lambda$

$X \rightarrow bS$

e.g one occurence of 000




$S \rightarrow X000X$

$S \Rightarrow 0S$

$S \rightarrow 1S$

$S \rightarrow \wedge$

e.g equal no of a's and b's.

$S \rightarrow aSbS$

$S \rightarrow aS1$

Single variable (LHS)

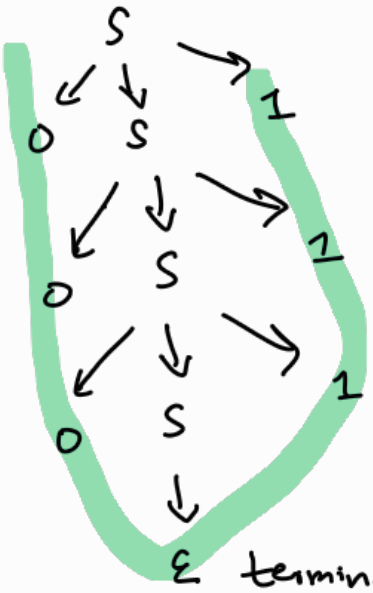
terminal or variables (S) — RHS

terminal: Symbols that cannot be broken down further, e.g. a and 1

$S \rightarrow \varepsilon$ → terminal

$S \rightarrow 0S1$

$S \rightarrow \varepsilon$

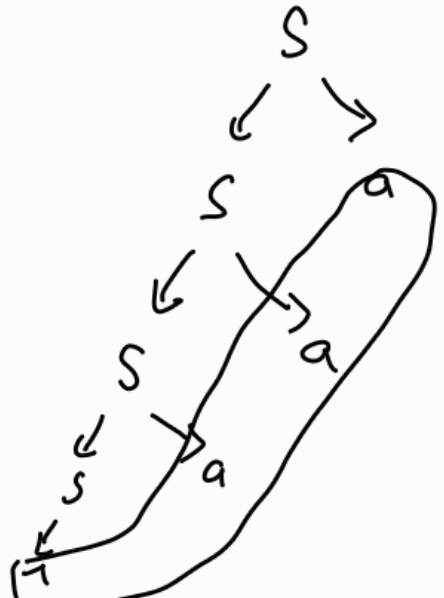



000ε111

000111 ✓

$S \rightarrow 1$

$S \rightarrow Sa$

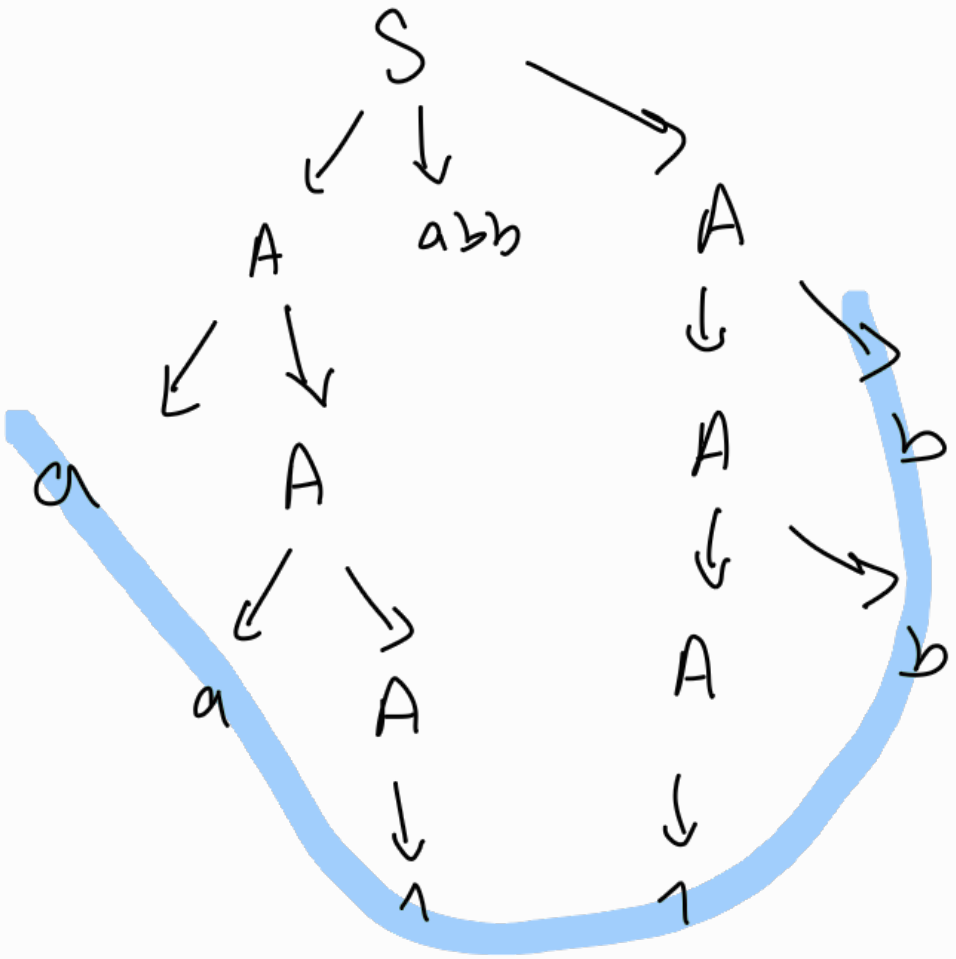


$(a+b)^*$ $abb$ $(a+b)^*$

A A

$S \rightarrow A\,abb\,A$

$A \rightarrow aA \mid bA \mid \wedge$



termination condition → min value

$a^{2n+3} b^n$ , $n \geq 0$

$a^{2(0)+3} b^{(0)} \rightarrow aaa$

$S \rightarrow aaa$ termination condition

we know $a^n b^n$

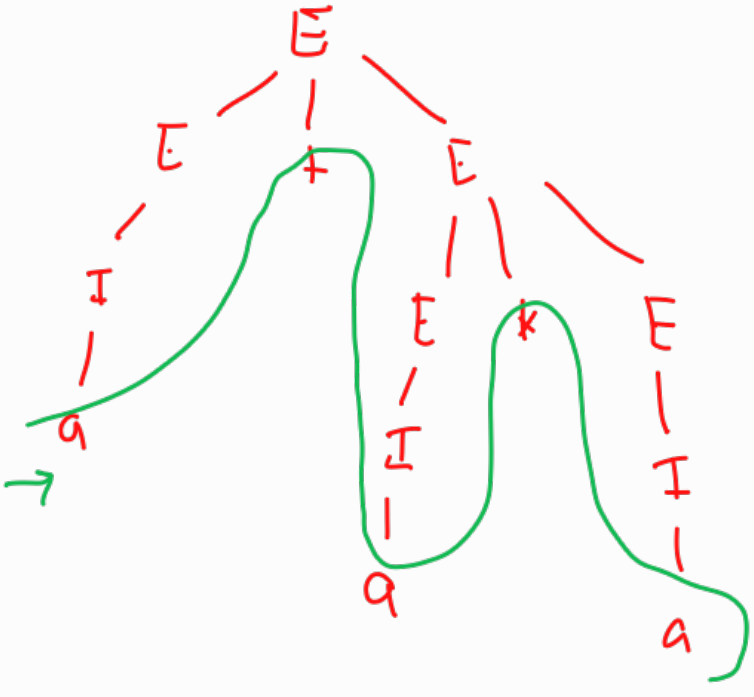
$S \rightarrow asb$

$S \rightarrow aaSb$

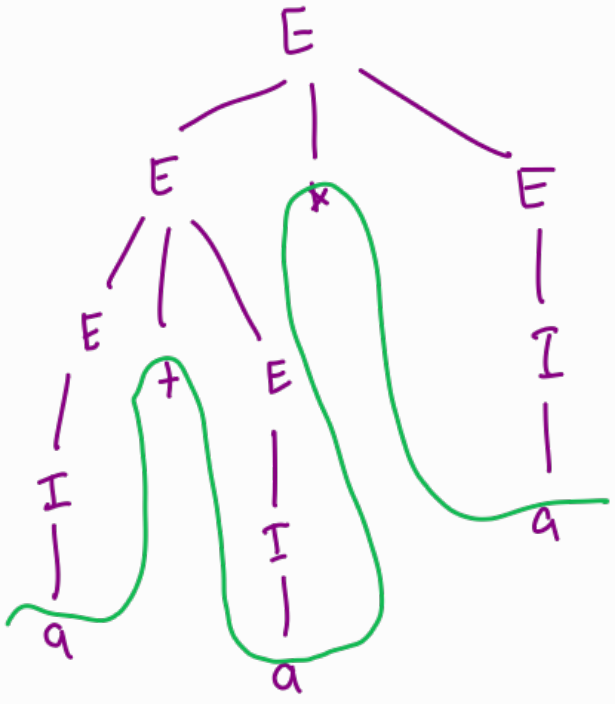
$S \rightarrow aaa$

---

Ambiguity:: happens if for 1 String we are able to make two diff trees that Both represent that same String




a + a * a




a + a * a

Two diff parse trees.

hence Ambiguous grammar

---

$L = \{ wcw^T \mid w \in \{a,b\}^* \}$

$\Lambda$, abcba, aaacaaa,

bababcabab

starting = ending

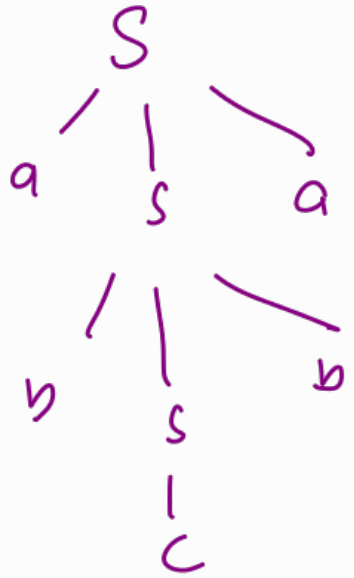
if starts with a

↳ $S \rightarrow aSa$

$S \rightarrow bSa$

if starts with b

$S \rightarrow c$



$L = \{ a^n b^m c^n \mid n, m \geq 1 \}$

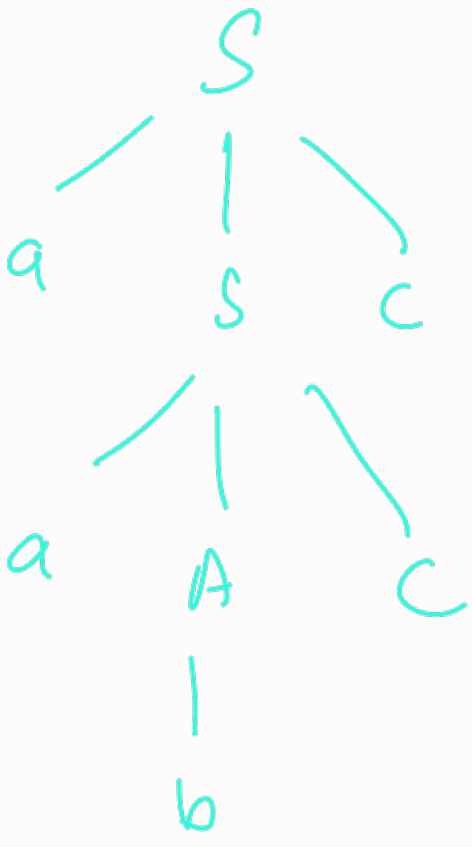
a and c are equal

aabcc

$b \to 1$

$S \to aSc \mid aAC$

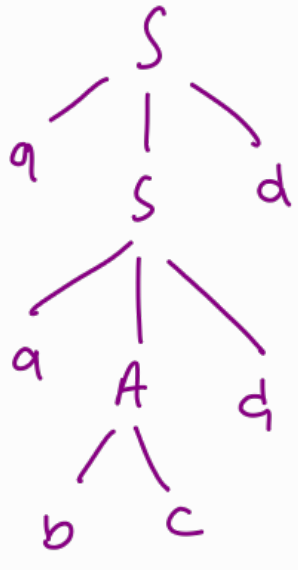
$A \to bA \mid b$




$L = \{ a^n b^m c^m d^n \mid n \geq 1, m \geq 1 \}$

$S \to aSd \mid aAd$

$A \to bAC \mid bC$




aabcdd

$L = \{ a^n b^n c^m d^m \mid n, m \geq 1 \}$

A B

$S \rightarrow AB$

$A \rightarrow aAb \mid ab$

$B \rightarrow cBd \mid cd$

$L = \{ a^{m+n} b^m c^n \mid n, m \geq 1 \}$

$a^n a^m b^m c^n$

$S \rightarrow aSc \mid aAc$

$A \rightarrow aAb \mid ab$

$S \to aAb$

$A \to aB \mid b$

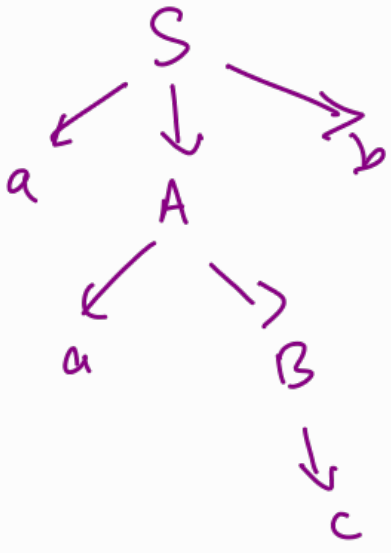
$B \to c$

start Symbol




Always generate the simplest string

abb

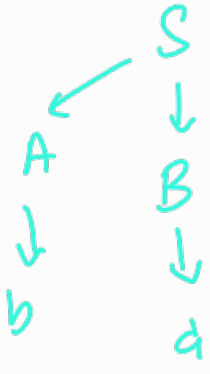


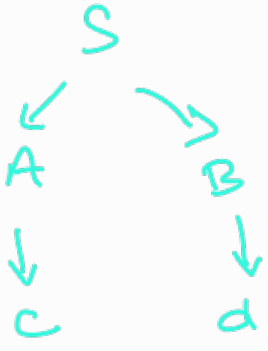

aacb

$S \to AB \mid Bb$

$A \to b \mid c$

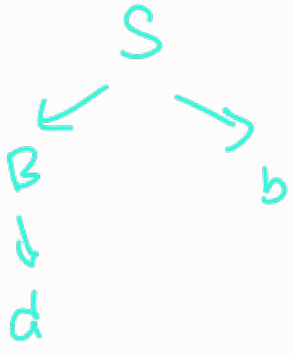
$B \to d$




bd

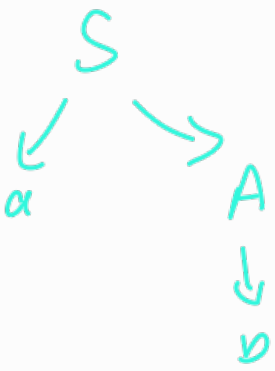
cd




db

$S \to aA \mid abS$

$A \to bS \mid b$




Smallest string: ab




abab

---

CFG → generates CFL that will be represented by pda.

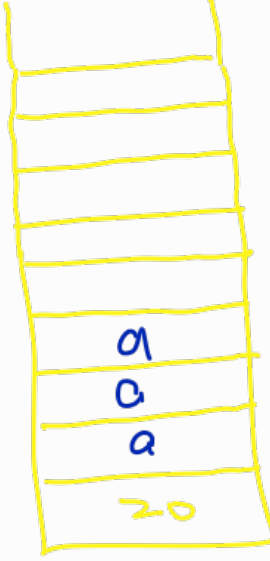
↓

non-deterministic

$L = \{a^n b^n \mid n \geq 1\}$

e.g. aaabbb

Push a and then for every b pop a.



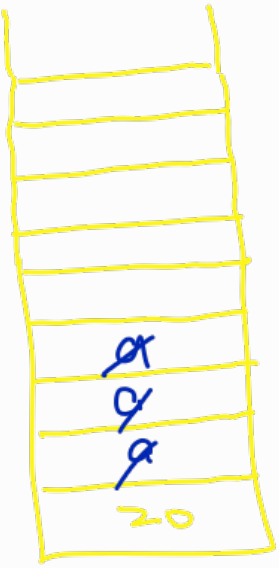

1) $a, Z0 \mid aZ0$
2) $a, a \mid aa$
3) $a, a \mid aa$

} push

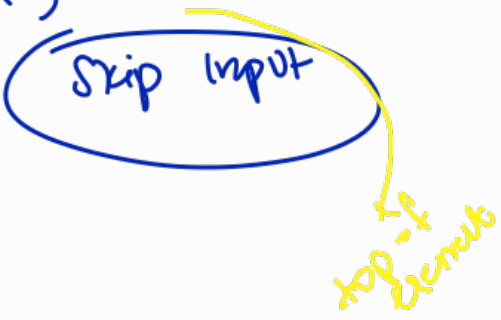
4) b, a \ ∧

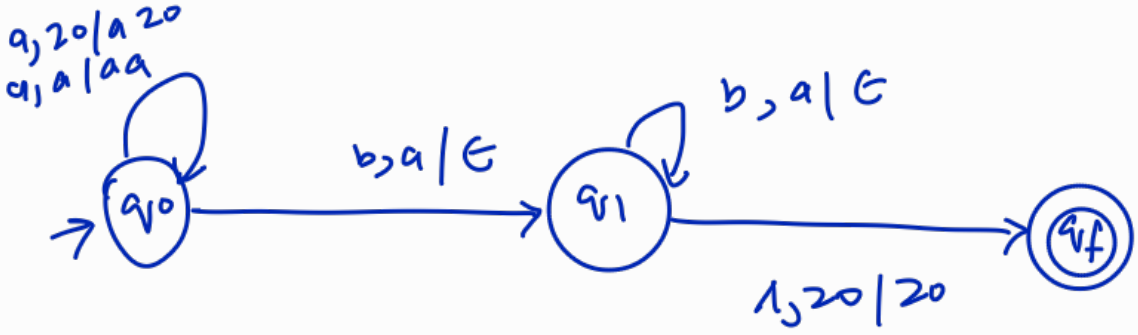
5) b, a \ ∧

6) b, a \ ∧




7) ∧, Z0 | Z0




(In stack we cannot push | pop two elements at one time)

→ ∧, Z0 | Z0 → replace Z0 with Z0

- ∧ : Input (consume no input)
- Z0 : top of stack

→ q, Z0 | a Z0

- q : Input
- Z0 : TOS
- a Z0 : Replace Z0 with a Z0

→ b, a | a → skip

- b : Input
- a : TOS
- a : Replace a with a

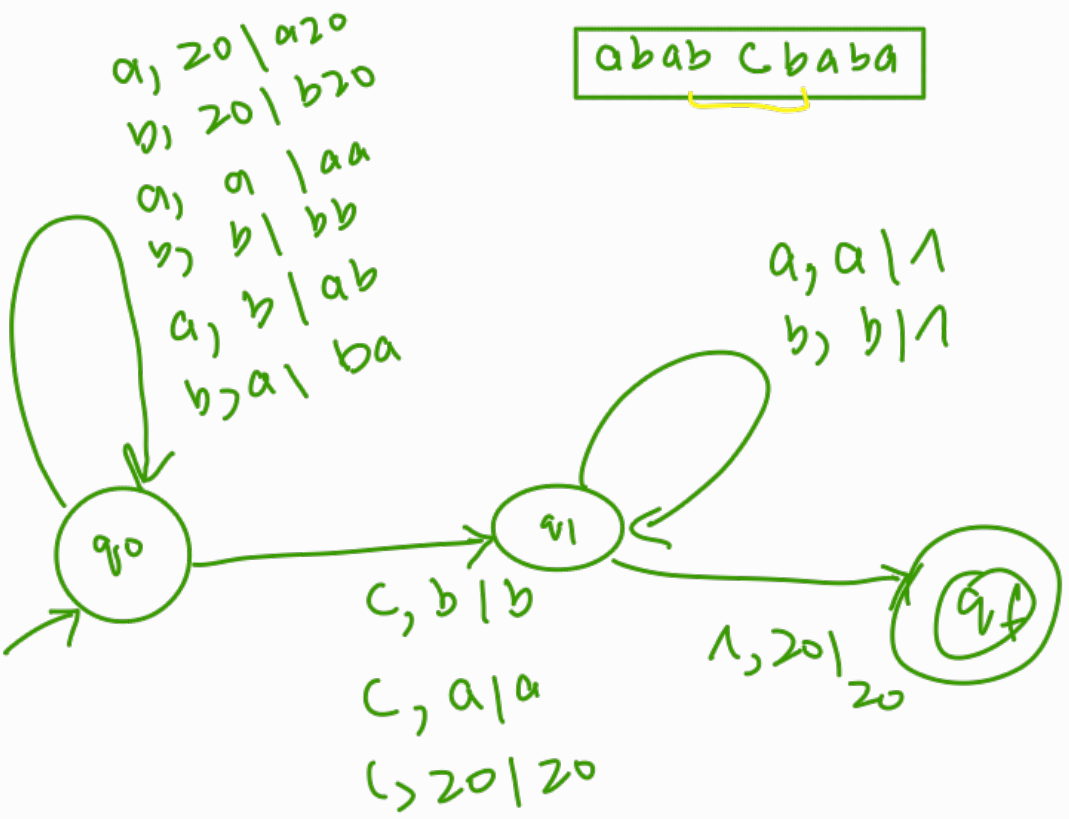
POP

q, Z0 | ∈

- q : Input
- Z0 : TOS
- ∈ : Replace Z0 with ∈

PDA for example:. $L = \{ wcw^R \mid w \in \{a, b\}^* \}$

abab cbaba

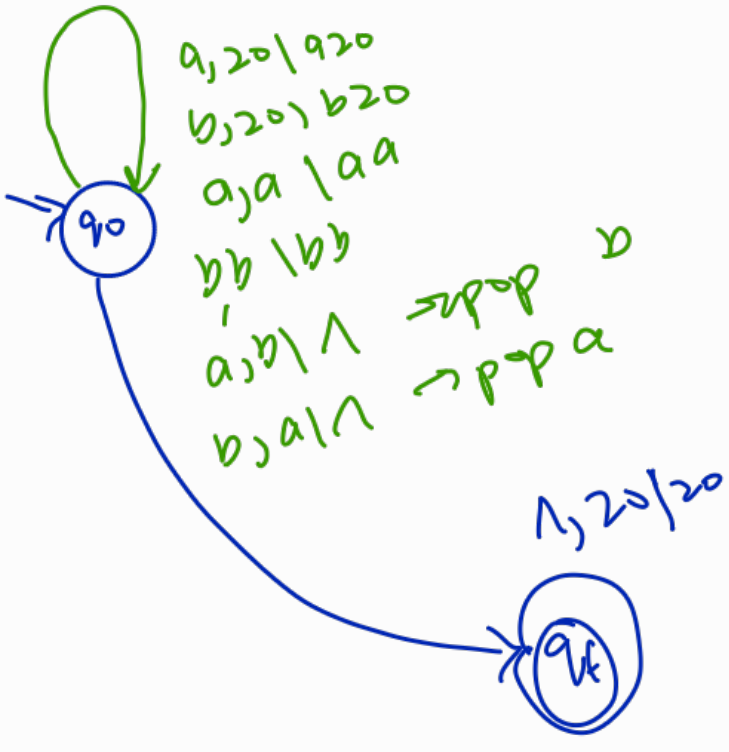






PDA for $L = \{ |w|_{a=b} \mid w \in (a, b)^* \}$

a b b b a bb aaa ∧

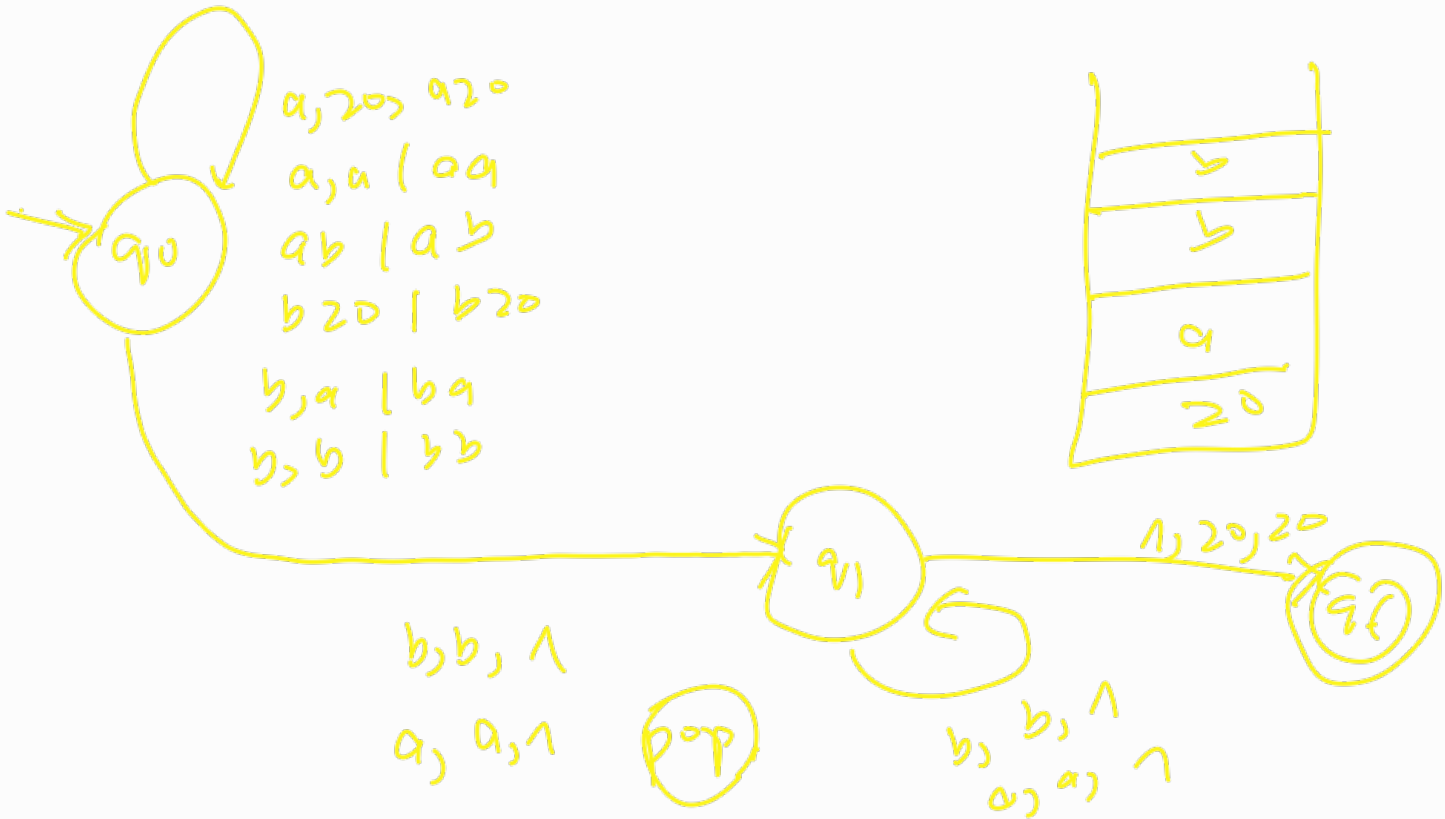



→ Logic:.

if a letter finds opposite on TOS → pop else Push itself

NPDA $L = \{ww^r \mid w \in (a,b)^*\}$

a bb bb a ^




---

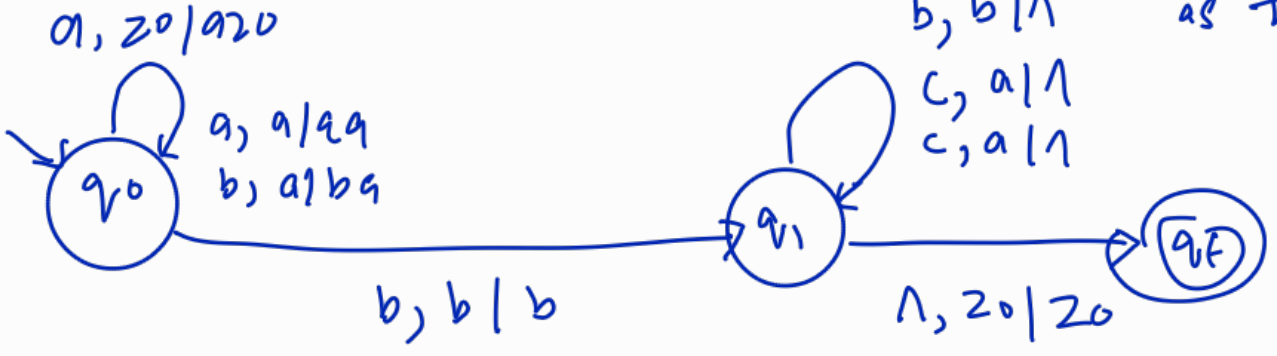
PDA for $L = \{a^n b^m c^n \mid n \geq 1, m \geq 1\}$

aa bbb cc ^

push pop

skip

for every a pushed pop c as they are equa



general Rule:: when two things match → pop , when they don't push into the Stack

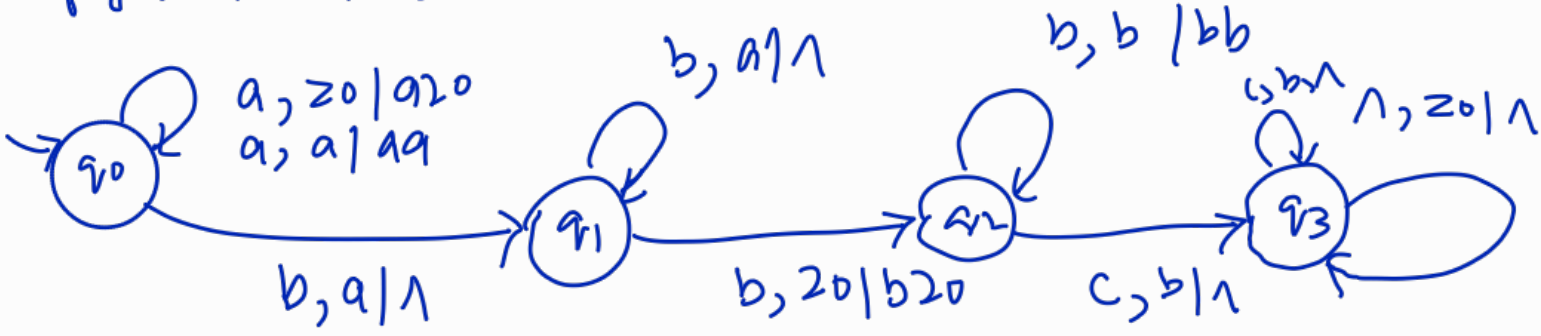
PDA for $L = a^n b^{m+n} c^m \mid m, n \geq 1$

$a^n + c^n = b^{m+n}$ for every a pushed pop b

e.g aa bbbbb ccc

$\rightarrow a^n b^n b^m c^m$

empty Stack → No Z0




$L = a^n b^{2n} \mid n \geq 1$ e.g aa bbbb

skip skip

push pop pop

logic:: for every 'a' pushed we are going to skip one b and push other.